left to deal with, and you should also write tests for each and every method, because it's easy to make mistakes in duplicated code.

"I can think of two different ways to remove this duplication," Bill says. "One is a spell called Dynamic Methods. The other is a special method called method_missing. We can try both solutions and decide which one we like better." You agree to start with Dynamic Methods and get to method_missing after that.

Dynamic Methods

Where you learn how to call and define methods dynamically, and you remove the duplicated code.

"When I was a young developer learning C++," Bill says, "my mentors told me that when you call a method, you're actually sending a message to an object. It took me a while to get used to that concept. If I'd been using Ruby back then, that notion of sending messages would have come more naturally to me."

Calling Methods Dynamically

When you call a method, you usually do so using the standard dot notation:

methods/dynamic_call.rb

```
class MyClass
  def my_method(my_arg)
    my_arg * 2
  end
end

obj = MyClass.new
obj.my_method(3)  # => 6
```

You also have an alternative: call MyClass#my_method using Object#send in place of the dot notation:

```
obj.send(:my_method, 3)   # => 6
```

The previous code still calls my_method, but it does so through send. The first argument to send is the message that you're sending to the object—that is, a symbol or a string representing the name of a method. (See *Method Names and Symbols*, on page 49.) Any remaining arguments (and the block, if one exists) are simply passed on to the method.

Why would you use send instead of the plain old dot notation? Because with send, the name of the method that you want to call becomes just a regular argument. You can wait literally until the very last moment to decide which